INT161 · Basic Backend Development

HTTP Basics

Node event loop, http module และการอ่าน method, URL, query และ path จาก request

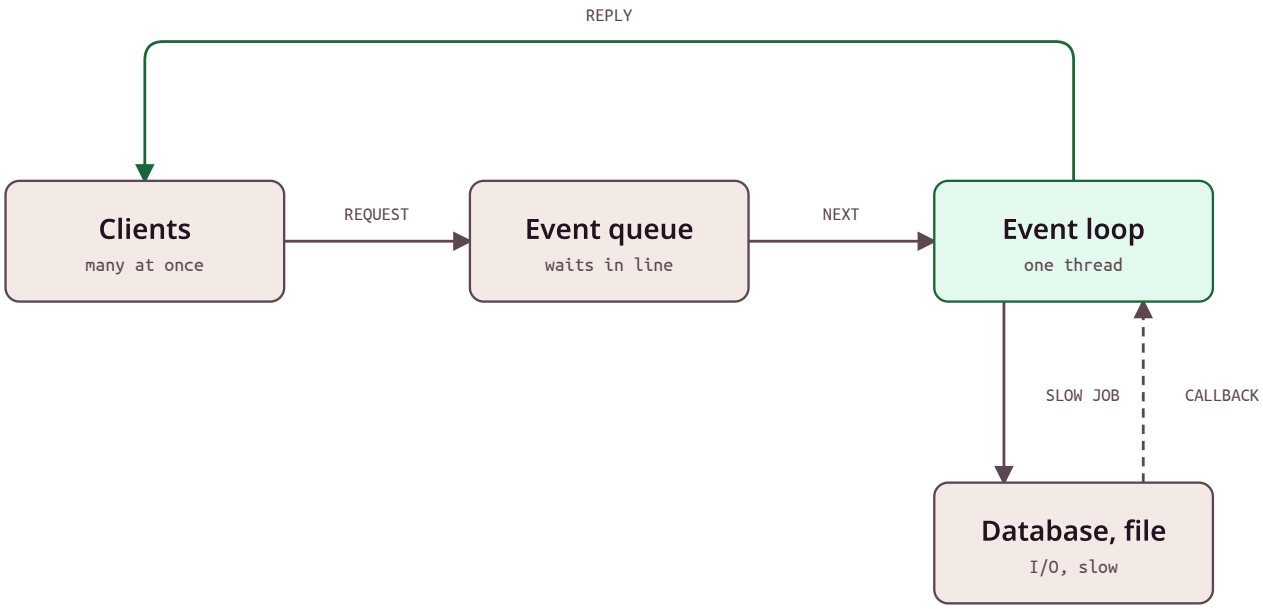
Node.js Architecture | Core Modules, CJS and ESM | The http Module | HTTP Methods | Query and Path Parameters | Exercise: Multiplication Table | อภิธานศัพท์

01

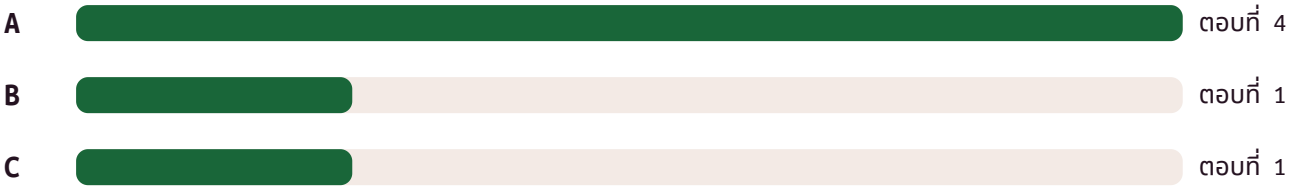
Node.js Architecture

เป้าหมายคลาสนี้: สร้าง **HTTP server** จัดการ **request** กับ **response** object และอ่าน **request parameter** กับ path

- input ของโปรแกรมคือ **HTTP** request และ output คือ HTTP response **Node** แปลงทั้งคู่เป็น object ให้ เราไม่ต้อง parse ข้อความเอง



ทีละอัน | Event queue



ลำดับที่ตอบ: B, C, A

A = ต้องรอ database · B = หน้าที่ส่งได้เลย · C = หน้าที่ส่งได้เลย

request ที่ช้าไม่ขวางอันอื่น ใครเสร็จก่อนได้ตอบก่อน

02

Core Modules, CJS and ESM

อาจารย์เปรียบ **built-in module** เหมือนคอนโดที่มีเฟอร์นิเจอร์ built-in มากับ Node และเอาออกไม่ได้

- built-in ไม่ได้แปลว่าใช้ได้เลย ยังต้อง import ก่อนใช้
- **module** ที่ไม่ได้ built-in ติดตั้งด้วย **npm**
- ที่ใช้บ่อย: fs สำหรับไฟล์, http สำหรับ **server**, path, os, events, util, **stream**, crypto, url, querystring
- คลาสนี้เน้น http เพราะเป็น HTTP server ที่คุยกับ **front end**

ESM (use this)

```
import * as http from 'node:http'
```

มาตรฐานทางการ ถูกอ่านล่วงหน้า โหลด module พร้อมกัน และทำ **tree shaking** ได้

CommonJS (legacy)

```
const http = require('node:http')
```

แบบเก่า โหลดทีละไฟล์ ตัวอย่างบนเว็บจำนวนมากยังใช้อยู่

> **ทำไมใช้ ESM**
>
> อาจารย์ถาม AI หลายตัวว่าควรใช้อะไร ทุกตัวตอบว่า ESM สำหรับโปรเจกต์ใหม่ ให้เขียน import ไว้บนสุดของไฟล์ ตัวอย่างบนเว็บอาจใช้ require ต้องแปลงเอง

03

The http Module

module นี้มีสี่ class หลัก สองตัวในนั้น Node สร้างให้ทุกครั้งที่มี request

`http.Server`

The server itself.

listen(**port**, **callback**) เริ่มทำงาน close() หยุดรับการเชื่อมต่อใหม่ event request เกิดทุกครั้งที่มี request

`http.IncomingMessage`

The request.

Node สร้างให้ อ่าน request.method, request.url และ request.**headers**

`http.ServerResponse`

The response.

Node สร้างให้เหมือนกัน writeHead() ตั้ง status และ header, write() เพิ่ม **body**, end() จบและส่ง

`http.ClientRequest`

A request you send.

ใช้เมื่อ **back end** ของเราเรียก server อื่น เช่น **API** ขนส่ง ตอนนั้นเราเป็น **client**

http-server.js

```
import * as http from 'node:http';
const listener = function (request, response) {
  const { url, method } = request;
  response.writeHead(200, { 'Content-Type': 'text/plain' });
  response.end(`You made a ${method} request to ${url}`);
};
const server = http.createServer(listener);
server.listen(3000, () => {
  console.log('Server running at http://127.0.0.1:3000/');
});
server.on((req, res) => {
  console.log('Request received:', req.method, req.url);
});
```

- handler รับสอง argument ลำดับตายตัว: request ก่อน response ทีหลัง ชื่อตั้งเองได้ ถ้าสลับชื่อ code จะพัง
- createServer แค่บอกว่าจะทำอะไร ยังไม่ทำงานจนกว่าจะ listen port เริ่มต้นในตัวอย่าง Node คือ 3000
- callback ของ listen คือวิธีรู้ว่า server เริ่มแล้ว ถ้าไม่มี server จะเงียบ
- end() จบและส่ง response ส่วน write() เรียกได้หลายครั้งก่อนหน้านั้น
- ถ้า Content-Type เป็น text/plain **browser** จะพิมพ์ HTML เป็นข้อความ ไม่ render

two terminals

```
# terminal 1
$ node http-server.js
Server running at http://127.0.0.1:3000/
# terminal 2: -i shows the response headers
$ curl -i http://localhost:3000/user
HTTP/1.1 200 OK
Content-Type: text/plain

You made a GET request to /user
```

> ⚠ **กับดัก port**
>
> สอง server ใช้ port เดียวกันไม่ได้ ถ้า node ยังรันอยู่แล้วเปิด bun ที่ port เดียวกัน bun จะรันโดยไม่ error แต่ไม่ได้รับอะไรเลย request ยังไปที่ node ตัวเก่า ทำให้เรา **debug** code ที่ไม่ได้รันอยู่ ต้องปิดตัวเก่าก่อน

04

HTTP Methods

ทุก request มี method และ **URL** อ่านได้ด้วย request.method และ request.url

GET

Read data

`/subjects, /students`

POST

Create a new resource

`/subjects (with data)`

PUT

Update existing data

`/subjects/{id} (with data)`

DELETE

Remove a resource

`/subjects/{id}`

- address bar ของ browser ส่งได้แค่ GET ถ้าจะทดสอบ POST หรือ PUT ใช้ **curl** หรือ **Postman**
- method สำคัญในภายหลัง ใน **REST** API การอ่านต้องใช้ GET และการสร้างต้องใช้ POST มีผลต่อการออกแบบ **endpoint**

curl with a method

```
$ curl -i -X PUT http://localhost:3000/user/5 -d '{"name": "John Doe", "role": "Admin"}'
You made a PUT request to /user/5
```

05

Query and Path Parameters

ข้อมูลเข้ามาใน URL สองแบบ ลองแยก URL ดูทีละชิ้น

/user?name=Somchai&fav_s...

/user/123

/multiplication-table/7

/multiplication-table?nu...

/user?name=Somchai&fav_subj=Back-end Dev&fav_subj=Front-end Dev

urlObj.pathname

"/user"

pathname.split('/')

["","user"]

.filter(Boolean)

["user"]

params.getAll('name')

["Somchai"]

params.getAll('fav_subj')

["Back-end Dev","Front-end Dev"]

split ตรง / ตัวแรกจะได้ชิ้นว่าง filter(Boolean) จึงตัดทิ้ง ชื่อที่ไม่ได้ส่งมาจะได้ null ทุกค่าเป็น string แม้แต่ 7

Query string: ส่วนหลัง ? เขียนเป็น name=value คั่นด้วย & เหมือนการค้นหาใน Google ที่ q เก็บคำที่พิมพ์ ชื่อเดียวกันซ้ำได้ เช่น checkbox ให้ใช้ getAll แล้วจะได้ array

Reading the query string

```
const urlObj = new URL(request.url, `http://${request.headers.host}`);
const params = urlObj.searchParams;
const name = params.get('name');
const subjects = params.getAll('fav_subj');
response.write(`Name = ${name} Favorite Subjects: ${subjects}\n`);
```

Path: split **pathname** ด้วย / เครื่องหมายแรกทำให้ได้ชิ้นว่าง จึงต้อง filter ทิ้ง

Reading the path

```
const urlObj = new URL(request.url, `http://${request.headers.host}`);
const pathSegments = urlObj.pathname.split('/').filter(Boolean);
// /users/123  ->  ['', 'users', '123']  ->  filter(Boolean)  ->  ['users', '123']
```

- ชื่อต้องตรงเป๊ะ ชื่อที่สะกดผิดคือคนละชื่อ และจะได้ null
- ทีม front end กับ back end ควรตกลงชื่อกันก่อนเริ่มงาน
- **framework** ทำให้ภายหลัง เรียนแบบเขียนเองสักครั้งจะได้รู้ว่ามันทำอะไรอยู่

06

Exercise: Multiplication Table

โจทย์ของอาจารย์: ตอบเป็นตารางสูตรคูณ output เป็นข้อความธรรมดาได้ หน้าตาไม่สำคัญ

- รับตัวเลขได้สองแบบ: /multiplication-table?number=7 หรือ /multiplication-table/7
- ถ้าส่งมาทั้งสองแบบ ให้เอาแบบ path
- ถ้า path ไม่ใช่ /multiplication-table ให้ตอบ status 400 และไม่ต้องมี body
- ทุก input เป็น string ต้องแปลงก่อนคูณ

> **วิธีฝึก**
> ใส่กฎเพิ่มแม้ตัวอย่างจะตอบแค่ 200 ฝึกคือสระว่ายน้ำ สอบคือทะเล พิมพ์เองอย่า paste ถ้าติดให้ถาม AI ว่า error แปลว่าอะไร อย่าให้เขียนทั้งโปรแกรม ตรวจแค่สิ่งที่ spec บอก เช่น input สองแบบ อย่าทำเกิน

คำตอบหนึ่งแบบที่ใช้ได้ ไว้เทียบหลังจากลองทำเองแล้ว

multiplication-table.js (one possible answer)

```
import * as http from 'node:http';
const listener = (request, response) => {
  const url = new URL(request.url, `http://${request.headers.host}`);
  const [route, pathNumber] = url.pathname.split('/').filter(Boolean);
  if (route !== 'multiplication-table') {
    response.writeHead(400);
    return response.end();
  }
  const n = Number(pathNumber ?? url.searchParams.get('number'));
  let table = '';
  for (let i = 1; i <= 12; i++) table += `${n} x ${i} = ${n * i}\n`;
  response.writeHead(200, { 'Content-Type': 'text/plain' });
  response.end(table);
};
http.createServer(listener).listen(3000);
```

07

อภิธานศัพท์

API
: Application Programming Interface: ช่องทางที่โปรแกรมเปิดให้โปรแกรมอื่นเรียกใช้

back end
: ส่วนบน server จัดการข้อมูล กฎ และ database แล้วตอบ front end ด้วย JSON

body
: ข้อมูลที่ request พกมาหลัง header สำหรับ API มักเป็น JSON

browser
: client ที่คนส่วนใหญ่ใช้ เช่น Chrome, Edge, Safari

built-in module
: module ที่มากับ Node ใช้ได้ทันที ไม่ต้องติดตั้ง

callback
: ฟังก์ชันที่ส่งไปให้เรียกทีหลังเมื่องานเสร็จ

client

ฝั่งที่ร้องขอ มักเป็น browser แต่เป็นโปรแกรมหรืออุปกรณ์อะไรก็ได้

curl

client แบบ command line ส่ง HTTP method ใดก็ได้ ใช้ทดสอบ POST และ PUT ได้ ใส่ -i เพื่อดู header ของ response

debug

หาและแก้บั๊ก มักด้วยการไล่ code ทีละบรรทัดและดูค่า

endpoint

method กับ path หนึ่งคู่ที่ API ตอบ เช่น GET /api/subjects

ESM

ระบบ module สมัยใหม่ ใช้ import และ export ต้องมี "type": "module" ใน package.json

framework

ชุดเครื่องมือที่ช่วยให้งานง่ายขึ้น แต่มีกรอบที่เราต้องเขียนตาม

front end

ส่วนที่ผู้ใช้เห็นและใช้ รันบน browser หรือโทรศัพท์

headers

ข้อมูลเสริมของข้อความ ส่งเป็นบรรทัด name: value เช่น Content-Type

HTTP

HyperText Transfer Protocol: ภาษาข้อความธรรมดาที่ browser กับ web server ใช้คุยกัน ฝั่งหนึ่งส่ง request อีกฝั่งส่ง response

HTTP server

ซอฟต์แวร์บนเครื่อง server ที่รับ HTTP request และส่ง response

module

ไฟล์ code ที่โหลดเข้าไฟล์อื่นด้วย import

Node

runtime ที่รัน JavaScript นอก browser ทำให้ใช้ภาษาเดียวกันเขียน back end ได้

npm

Node Package Manager: ติดตั้ง library เช่น express ลงใน node_modules

pathname

ส่วนของ URL หลัง host และก่อน ? เช่น /user/123

port

เลขที่บอกว่าโปรแกรมไหนบนเครื่องควรได้รับข้อความ web server ใช้ 80 เป็นค่าเริ่มต้น

Postman

เครื่องมือส่ง HTTP method ใดก็ได้ไปที่ API ใช้ทดสอบก่อนที่ front end จะเสร็จ

Query string

ส่วนของ URL หลัง ? เช่น page=2&size=10 ใช้กรองหรือแบ่งหน้ารายการ

request

สิ่งที่ client ส่งมา: method, URL, header และบางครั้งมี body

request parameter

คู่ name=value หลัง ? ใน URL หลายตัวคั่นด้วย & ทุกค่าที่ได้เป็น string

response

สิ่งที่ server ส่งกลับ: status code, header และ body

REST

แนวทางออกแบบ web API: resource ตั้งชื่อด้วย URL และเลือก action ด้วย HTTP method

server

โปรแกรม (หรือเครื่องที่รันมัน) ที่รอรับ request แล้วตอบกลับ

stream

ข้อมูลที่ทยอยมาเป็นชิ้นแทนที่จะมาพร้อมกัน

tree shaking

ขั้นตอน build ที่ตัด code ที่ import มาแต่ไม่ได้ใช้ ESM ทำได้ CJS ทำไม่ได้

URL

ที่อยู่ของ resource ประกอบด้วย protocol, host, port และ path เช่น http://localhost:3000/user